

Normalization Process (Pre-requisite):

Originally, the individual World Bank datasets were not normalized because they were in wide format; each row had a country matched with two year values. To normalize the data, I used Excel and Power Query's unpivot functionality to transform the individual World Bank datasets into long format; this meant that every row had a country matched with only one year value. This transformed the datasets into First Normal Form (1NF), since each field contains a single atomic value and the repeating year columns were eliminated. The resulting long-format CSV files were then imported into PostgreSQL, where SQL was used for further cleaning, validation, and table joins. I normalized the data for each of the four tables (GDP per capita, health expenditure per capita, death rate, and population age 65+), and I used a composite key consisting of country_code and year to uniquely identify each row. I then joined these four tables using that composite key.

SQL Queries:

1. Identifying duplicates - The following syntax checks if duplicates exist, and it checks which (country_code, year) combinations are duplicated (ask GPT to explain logic on this a bit more)

This code is for the pop_age_65_plus table - since this SQL query returned no rows, that means there are no duplicates! So, I will move onto checking for duplicates for the remaining tables.

```
SELECT country_code,year,COUNT(*)  
FROM pop_age_65_plus  
GROUP BY country_code,year  
HAVING COUNT(*) > 1;
```

This code is for the gdp_per_capita table - since this SQL query returned no rows, that means there are no duplicates! So, I will move onto checking for duplicates for the remaining tables.

```
SELECT country_code,year,COUNT(*)  
FROM gdp_per_capita  
GROUP BY country_code,year  
HAVING COUNT(*) > 1;
```

This code is for the death_rate table - since this SQL query returned no rows, that means there are no duplicates! So, I will move onto checking for duplicates for the remaining tables.

```
SELECT country_code,year,COUNT(*)  
FROM death_rate  
GROUP BY country_code,year  
HAVING COUNT(*) > 1;
```

This code is for the current_health_expenditure_per_capita table - since this SQL query returned no rows, that means there are no duplicates! So, I will move onto checking for duplicates for the remaining tables.

```
SELECT country_code,year,COUNT(*)
FROM current_health_expenditure_per_capita
GROUP BY country_code,year
HAVING COUNT(*) > 1;
```

So, my tables don't have any duplicate values!

1. Handling Null Values - we have a composite join key - need both country_code AND year to uniquely identify rows!

- a) One row per country per year - so check if any rows are missing the composite join key

```
SELECT *
FROM pop_age_65_plus
WHERE country_code IS NULL
OR year IS NULL;
```

I repeated that code for all the other tables (just replace the table name following FROM clause)
- found that there were no rows that were missing the composite join key!

```
SELECT *
FROM gdp_per_capita
WHERE country_code IS NULL
OR year IS NULL;
```

```
SELECT *
FROM death_rate
WHERE country_code IS NULL
OR year IS NULL;
```

```
SELECT *
FROM current_health_expenditure_per_capita
WHERE country_code IS NULL
OR year IS NULL;
```

- b) Check missing values in the data columns

```
SELECT COUNT(*) AS total_rows,
COUNT(country_code) AS nonnull_country_code,
COUNT(year) AS non_null_year,
COUNT(pop_65_plus) AS nonnull_pop_65_plus,
COUNT(*) - COUNT(pop_65_plus) AS missing_pop65

FROM pop_age_65_plus;
```

For above command for pop_age_65_plus table, I found that total_rows, non_null_country_code, non_null_year, and non_null_pop_65_plus all had 434 rows - so there were NO NULL VALUES! - **perfect dataset - also the missing_pop65 column has 0 rows, so that means no null values in the columns!**

```
SELECT COUNT(*) AS total_rows,  
COUNT(country_code) AS nonnull_country_code,  
COUNT(year) AS non_null_year,  
COUNT(gdp_per_capita) AS nonnull_gdp_per_capita,  
COUNT(*) - COUNT(gdp_per_capita) AS missing_gdp_per_capita
```

```
FROM gdp_per_capita;
```

For above command for gdp_per_capita table, I found that total_rows, non_null_country_code, and non_null_year had 434 rows - BUT nonnull_gdp_per_capita had 415 rows - **19 missing values - the missing_gdp_per_capita column has 19 rows**

```
SELECT COUNT(*) AS total_rows,  
COUNT(country_code) AS nonnull_country_code,  
COUNT(year) AS non_null_year,  
COUNT(death_rate) AS nonnull_death_rate,  
COUNT(*) - COUNT(death_rate) AS missing_death_rate
```

```
FROM death_rate;
```

For above command for death_rate table, I found that total_rows, non_null_country_code, non_null_year, and non_null_death_rate all had 434 rows - so there were NO NULL VALUES! - **perfect dataset - the missing_death_rate column has 0 rows**

```
SELECT COUNT(*) AS total_rows,  
COUNT(country_code) AS nonnull_country_code,  
COUNT(year) AS non_null_year,  
COUNT(current_health_expenditure_per_capita) AS  
nonnull_current_health_expenditure_per_capita,  
COUNT(*) - COUNT(current_health_expenditure_per_capita) AS  
missing_current_health_expenditure_per_capita
```

```
FROM current_health_expenditure_per_capita;
```

For above command for current_health_expenditure_per_capita table, I found that total_rows, non_null_country_code, and non_null_year had 434 rows - BUT nonnull_current_health_expenditure_per_capita had 377 rows - **57 missing values - the missing_current_health_expenditure_per_capita column has 57 rows**

So for all the tables, the join keys do NOT have any missing values! So no NULL values for country_code or year (the composite join key) - so can join all the tables without losing rows due to join key issues

Now, I have to decide between keeping all rows (allowing missing values) OR keeping only complete cases (i.e. drop rows where ANY variable is missing) → The better choice is to drop rows that have null values, as this will not skew the regression results as much!

So, the next step is to now join the datasets together - build the combined dataset!

SQL code for creating the combined dataset - this is a view (NOT a physical table, but just storing the query - like a saved SQL query) - this view includes the null values (NOT yet cleaned/removed the null values)

```
CREATE VIEW combined_data AS
SELECT
  p.country_code,
  p.country_name,
  p.year,
  p.pop_65_plus,
  g.gdp_per_capita,
  h.current_health_expenditure_per_capita,
  d.death_rate
FROM pop_age_65_plus p
LEFT JOIN gdp_per_capita g
  ON p.country_code = g.country_code
  AND p.year = g.year
LEFT JOIN current_health_expenditure_per_capita h
  ON p.country_code = h.country_code
  AND p.year = h.year
LEFT JOIN death_rate d
  ON p.country_code = d.country_code
  AND p.year = d.year;
```

Troubleshoot this part after I now fixed gdp_per_capita CSV file with 2000 instead of 2020 as year typo

Next, I will confirm that I properly created the view. I will use the following SQL code:

```
SELECT * FROM combined_data LIMIT 10;
```

As expected, this query outputted the first 10 rows.

Step 3 is to check the years are correct.

To do this, I will use the following SQL query:

```
SELECT DISTINCT year
FROM combined_data
ORDER BY year;
```

The result is 2000 and 2019, so this ensures I have the correct years for the data.

Step 4 is to check the row count - ensure it is still 434 rows.

```
SELECT COUNT(*) FROM combined_data;
```

The result is 434 rows.

Step 5 is to now, check for missing values AFTER we joined the data sets!

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(gdp_per_capita) AS nonnull_gdp,
  COUNT(current_health_expenditure_per_capita) AS nonnull_health,
  COUNT(pop_65_plus) AS nonnull_pop65,
  COUNT(death_rate) AS nonnull_death
FROM combined_data;
```

This time, we get the following output: total_rows as 434, nonnull_gdp as 415, nonnull_health as 377, nonnull_pop65 as 434, and nonnull_death as 434.

Step 6 - check for rows where GDP and health are present using the following SQL query:

```
SELECT COUNT(*) AS complete_cases
FROM combined_data
WHERE gdp_per_capita IS NOT NULL
AND current_health_expenditure_per_capita IS NOT NULL
AND pop_65_plus IS NOT NULL
AND death_rate IS NOT NULL;
```

The output is 374 rows. That means we retained 374 rows out of the 434 rows total, so there is

86% of the data retained. (very different from the 44% retention with 190 rows retained that I got when I messed up 2020 vs 2000 in the date! - issue was year mismatch, NOT actual missing data!)

So, 374 complete cases means there are 374 rows where GDP is present, health expenditure is present, population 65+ is present, and death rate is present → so there is a large, clean, fully observable dataset.

Step 7 - checking how many rows per year

```
SELECT
```

```
year,  
COUNT(*) AS total_rows,  
COUNT(gdp_per_capita) AS nonnull_gdp,  
COUNT(current_health_expenditure_per_capita) AS nonnull_health  
FROM combined_data  
GROUP BY year  
ORDER BY year;
```

Output: 217 total rows for 2000, 204 nonnull_gdp rows, 184 nonnull_health rows
217 total rows for 2019, 211 nonnull_gdp rows, 193 nonnull_health rows

I didn't check for death_rate and pop_age_65_plus because those were perfect data sets with no missing values.

So for 2000, GDP is missing $217 - 204 = 13$ rows. For health expenditure, $217 - 184 = 33$ rows are missing.

For 2019, GDP is missing $217 - 211 = 6$ rows. For health expenditure, $217 - 193 = 24$ rows are missing.

So health expenditure in 2000 is the main limiting variable (most rows missing with 33).

To do 2000 vs. 2019 comparison, we can use a balanced panel with non-null predictors because this gives an apples to apples comparison with a consistent sample.

For the next part, I will build the correct dataset to do the comparison.

1. Keeping both the years AND non-null predictors

```
CREATE VIEW valid_rows AS  
SELECT *  
FROM combined_data
```

```
WHERE YEAR in (2000,2019)
AND gdp_per_capita IS NOT NULL
AND current_health_expenditure_per_capita IS NOT NULL
AND pop_65_plus IS NOT NULL;
```

The output is a table with 374 rows that are loaded. These are valid observations, but these are NOT fully balanced - i.e. this includes rows where some countries appear only in 1 year.

2. Keeping only countries in BOTH years - some countries appear in both years (e.g. USA 2000, USA 2019) whereas some countries appear in only one year (e.g. Chile 2000, but there is no Chile 2019) → My goal is to get countries with valid data in BOTH 2000 AND 2019! → so in that case I want to include USA but exclude Chile

I need to ensure we only keep countries in BOTH years because otherwise, the averages would be comparing 1 set of countries in 2000 and another set of countries in 2019 - so there would not be an apples-apples comparison/lack of consistency!

```
CREATE VIEW balanced_panel AS
SELECT *
FROM valid_rows
WHERE country_code IN (
SELECT country_code
FROM valid_rows
GROUP BY country_code
HAVING COUNT(DISTINCT year) = 2
);
```

The output is a table with 364 rows that are loaded. So I removed the rows where country appears in only 1 year, so going from 374 rows before to 364 rows after this query means that there were 10 rows (countries) with data in only one year.

3. Check final size

```
SELECT COUNT(DISTINCT country_code) FROM balanced_panel;
```

Output is 182 rows (just the number). Since each country has 2 rows, there are 182 distinct countries because $182 \times 2 = 364$ (which matches the 364 rows from the previous query in step 2).

Last part, feature engineering.

1. Log transformations

```

CREATE VIEW balanced_panel_features AS
SELECT
*,
LN(gdp_per_capita) AS log_gdp,
LN(current_health_expenditure_per_capita) AS log_health
FROM balanced_panel
WHERE gdp_per_capita > 0
AND current_health_expenditure_per_capita > 0;

```

I am choosing this feature because GDP per capita and health expenditure per capita are very skewed when looking at the raw data, and using logs will reduce skew and variance. As a result, it is easier to create linear relationships, and that allows for a smoother regression analysis. Additionally, I know that economists often use log transformations, so doing this is standard practice for regression analysis.

2. Change over time from 2000 to 2019 in dependent and predictor variables for each country (e.g. change in GDP per capita, health expenditure per capita, population age 65+, and death rates)

```

CREATE VIEW country_changes AS
SELECT

a.country_code,
a.death_rate AS death_2000,
b.death_rate AS death_2019,
b.death_rate - a.death_rate AS death_change,

a.gdp_per_capita AS gdp_2000,
b.gdp_per_capita AS gdp_2019,
b.gdp_per_capita - a.gdp_per_capita AS gdp_change,

a.current_health_expenditure_per_capita AS health_2000,
b.current_health_expenditure_per_capita AS health_2019,
b.current_health_expenditure_per_capita - a.current_health_expenditure_per_capita AS
health_change,

a.pop_65_plus AS pop65_2000,
b.pop_65_plus AS pop_65_2019,
b.pop_65_plus - a.pop_65_plus AS pop_65_change

FROM balanced_panel a
JOIN balanced_panel b
ON a.country_code = b.country_code
WHERE a.year = 2000

```


AND b.year = 2019;

I want to create this feature because the overarching purpose of the project is to compare mortality rates between 2000 and 2019 and how the predictor variables of GDP per capita, health expenditure per capita, and population age 65+ are driving those changes in mortality rates over time. So, adding this feature allows us to directly see how changes in economic conditions, healthcare investment, and demographic structure influence changes in mortality rates over time.

I am choosing the balanced panel dataset with 364 rows/observations (only data points where each country has data in both years - so 182 countries observed in both 2000 and 2019) vs the dataset with 374 rows (which includes data points where some country have data in only 1 year) because I think consistency is more important than a small increase in sample size.

Restricting the dataset to countries that have data in both years ensures consistency when we are comparing the changes in the dependent variable and predictors over time (i.e. we are comparing changes for the same country in both 2000 and 2019). If we used the dataset with 374 rows instead, we might end up comparing changes in the dependent and predictor variables for 2 different countries over time (e.g. comparing US in 2000 vs. India in 2019). So, even though the sample size might increase by 10 observations, this benefit is largely outweighed by the detrimental impact of inconsistent comparisons.